KU LEUVEN

FACULTEIT INGENIEURSWETENSCHAPPEN

Verslag: Dynamic parallelism and and asynchronous operation

Geavanceerde computerarchitectuur

Robbe Hovaert
Lukas Hoste

Master in de ingenieurswetenschappen
Academiejaar 2024-2025

Context

Het doel van dit labo is om kennis te maken met dynamic parallelism en asynchrone operaties op de GPU. Dynamic parallelism wordt niet uitgevoerd omdat dit niet op alle GPU's ondersteund is. Om asynchrone operaties te testen wordt ten eerste reductie op vier arrays synchroon uitgevoerd en vervolgens asynchroon.

Methodologie

Synchroon

In de synchrone versie wordt ten eerste een kopie gemaakt van de meegegeven array (om zo eigenlijk vier arrays te bekomen). Er wordt hiervoor in een for loop telkens een memcpy gedaan om de nieuwe array te maken waarna de cpu "gpuAdvancedMaxReductionSequental" opgeropen wordt. Hierin wordt de array naar een macht van twee gebracht waarna een gewone cudaMemcpy gedaan wordt en de kernel gestart wordt.

Asynchroon

In de asynchrone versie wordt direct de array vergroot tot een macht van twee. We maken gebruik van CUDA streams om alle operaties asynchroon te doen, met dan code die per stream een andere operatie uitvoert. Wij gebruiken overal de "Async" functievarianten voor alloc, free, memcpy, ... De kernel krijgt de nodige stream ook mee. Er wordt pas geblockt vanaf wij het geheugen terug kopiëren naar de host.

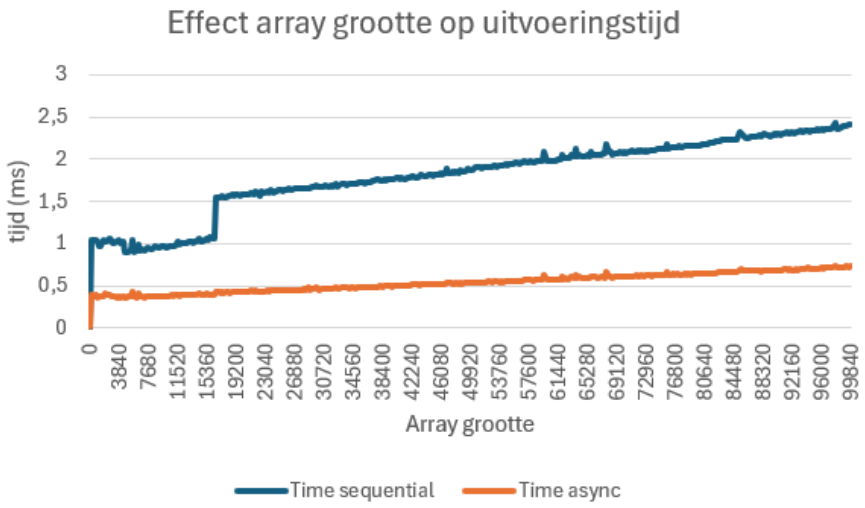
Observaties en resultaten

Wij hebben aan de bovenstaande implementatie timing-informatie toegevoegd. Wij houden de tijd dat de CPU nodig heeft om de CPU functies uit te voeren bij (TimeAsync en TimeSeq).

Om het effect van de asynchrone uitvoering te testen worden beide methoden uitgevoerd met verschillende array groottes. uit deze testen zien we het verwachte resultaat (Figuur 1). De tijden van beide methoden stijgen quasi-lineair, maar de synchrone stijgt iets sterker. Dit is te verwachten omdat bij de asynchrone versie al het setup werk tegelijk gebeurd en er dus minder gewacht moet worden waardoor de extra generatietijd minder opstapelt.

Conclusie

In dit labo hebben wij het belang onderzocht van het gebruik van asynchrone operaties voor (setup) werk op de CPU. We merkten een relatief groot verschil in performantie tussen een synchrone en een asynchrone variant. Met behulp van CUDA streams was de asynchrone versie eenvoudig om te implementeren. Hieruit kunnen we besluiten dat CUDA asynchroon gebruiken zeker de moeite is omdat de aanpassing om dit zo te krijgen eenvoudig is en voor een grote performantieverbetering zorgt.



Figuur 1: Array grootte tegenover executietijd in ms (gemiddelde van 50 uitvoeringen)

A CODE

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <chrono>

#define N 100000
#define MAX_BLOCK_SIZE 1024

void printArr(int* arr, int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

__device__ void add_fn(int* dst, int src1, int src2) {
    if (src1 == INT_MIN) {
        *dst = src2;
    } else if (src2 == INT_MIN) {
        *dst = src1;
    } else {
    *dst = src1 + src2;
    }
}

__device__ void min_fn(int* dst, int src1, int src2) {
    if (src1 == INT_MIN) {
        *dst = src2;
    } else if (src2 == INT_MIN) {
        *dst = src1;
    } else {
    *dst = min(src1, src2);
    }
}

__device__ void max_fn(int* dst, int src1, int src2) {
    *dst = max(src1, src2);
}

__device__ void mult_fn(int* dst, int src1, int src2) {
    if (src1 == INT_MIN) {
        *dst = src2;
    } else if (src2 == INT_MIN) {
        *dst = src1;
    } else {
    *dst = src1 * src2;
```

```
    }
}



__global__ void __gpuAdvancedMaxReduction(int* arr, int* output, int n,
↪  int operationMode) {
    __shared__ int buf1[2048]; // max 1024 threads per block, so 2048 is
    ↪  enough
    __shared__ int buf2[2048];


    n = min(n, blockDim.x * 2); // we keep in our local block (the length
    ↪  of the small array)

    int* shiftedArray = arr + (blockIdx.x * blockDim.x * 2); // base of
    ↪  this block
    int index = threadIdx.x * 2;

    if(threadIdx.x == 0) {
        for (int i = 0; i < n; i++) {
            buf1[i] = shiftedArray[i];
        }
    }

    // sync threads
    __syncthreads();


    int* src = buf1;
    int* dst = buf2;


    // we always reduce to half the size
    while(n != 1) {
        if (index < n) { // out of bounds in this iteration, disable
            if (index + 1 < n) {
                if (operationMode == 0) {
                    max_fn(&dst[index / 2], src[index], src[index + 1]);
                } else if (operationMode == 1) {
                    min_fn(&dst[index / 2], src[index], src[index + 1]);
                } else if (operationMode == 2) {
                    add_fn(&dst[index / 2], src[index], src[index + 1]);
                } else if (operationMode == 3) {
                    mult_fn(&dst[index / 2], src[index], src[index + 1]);
                } else {
                    dst[index / 2] = -1;
                }
            } else {
```

```
                // possible last element when the array is not even. it
                ↪  is always "bigger"
                dst[index / 2] = src[index];
            }
        }


        int* temp = src;
        src = dst;
        dst = temp;
        // in in half with round up! (the round down by default
        ↪  counteracts the +1)
        n = (n + 1) / 2;
        __syncthreads();
    }

    if (threadIdx.x == 0) {
        output[blockIdx.x] = src[0];
    }
}


int gpuAdvancedMaxReductionSequental(int* arr, int n, int
↪  threadsPerBlock, int operationMode) {
    // we need to round up to the nearse MAX_BLOCK_SIZE
    int allignedN = (int)ceil((double)n / (double)(threadsPerBlock * 2))
    ↪  * threadsPerBlock * 2;
    int* allignedArr = (int*) malloc(allignedN * sizeof(int));

    for (int i = 0; i < n; i++) allignedArr[i] = arr[i];
    for (int i = n; i < allignedN; i++) allignedArr[i] = INT_MIN; // Use
    ↪  INT_MIN for safety

    int *gpuArr1, *gpuArr2;
    cudaMalloc(&gpuArr1, allignedN * sizeof(int));
    cudaMalloc(&gpuArr2, (allignedN ) * sizeof(int)); // Allocate only
    ↪  needed space

    cudaMemcpy(gpuArr1, allignedArr, allignedN * sizeof(int),
    ↪  cudaMemcpyHostToDevice);


    int currentN = allignedN;

    while(currentN > 1) {
        int amountBlocks = (int)ceil((double)currentN /
        ↪  (double)(threadsPerBlock * 2));
```

```
        __gpuAdvancedMaxReduction<<<amountBlocks,
        ↪  threadsPerBlock>>>(gpuArr1, gpuArr2, currentN,
        ↪  operationMode);
        cudaDeviceSynchronize();


        // each block became one value
        currentN = amountBlocks;


        int* temp = gpuArr1;
        gpuArr1 = gpuArr2;
        gpuArr2 = temp;
    }

    int maxVal;
    cudaMemcpy(&maxVal, gpuArr1, sizeof(int), cudaMemcpyDeviceToHost);

    cudaFree(gpuArr1);
    cudaFree(gpuArr2);

    const auto endCPU = std::chrono::steady_clock::now();
    float milliseconds = 0;
    free(allignedArr);


    return maxVal;
}

// does all of the operatios at once (so it is merged with the calling
↪  code)
float asyncTest(int* arr, int n, int threadsPerBlock, int*dst ) {
    const auto startCPU = std::chrono::steady_clock::now();

    // we need to round up to the nearse MAX_BLOCK_SIZE
    int allignedN = (int)ceil((double)n / (double)(threadsPerBlock * 2))
    ↪  * threadsPerBlock * 2;
    int* allignedArr = (int*) malloc(allignedN * sizeof(int));

    for (int i = 0; i < n; i++) allignedArr[i] = arr[i];
    for (int i = n; i < allignedN; i++) allignedArr[i] = INT_MIN; // Use
    ↪  INT_MIN for safety

    cudaStream_t stream[4];
    int* gpuArr1Pointers[4];
    int* gpuArr2Pointers[4];

    for (int i = 0; i < 4; i++) {
```

```
        cudaStreamCreate(&stream[i]);
        int *gpuArr1, *gpuArr2;
        cudaMallocAsync(&gpuArr1, allignedN * sizeof(int), stream[i]);
        cudaMallocAsync(&gpuArr2, allignedN * sizeof(int), stream[i]);
        cudaMemcpyAsync(gpuArr1, allignedArr, allignedN * sizeof(int),
        ↪  cudaMemcpyHostToDevice, stream[i]);

        int currentN = allignedN;

        while(currentN > 1) {
            int amountBlocks = (int)ceil((double)currentN /
            ↪  (double)(threadsPerBlock * 2));
            __gpuAdvancedMaxReduction<<<amountBlocks, threadsPerBlock, 0,
            ↪  stream[i]>>>(gpuArr1, gpuArr2, currentN, i);
            // each block became one value
            currentN = amountBlocks;


            int* temp = gpuArr1;
            gpuArr1 = gpuArr2;
            gpuArr2 = temp;
        }
        gpuArr1Pointers[i] = gpuArr1;
        gpuArr2Pointers[i] = gpuArr2;
    }

    for (int i = 0; i < 4; i++) {
        cudaMemcpyAsync(dst + i, gpuArr1Pointers[i], sizeof(int),
        ↪  cudaMemcpyDeviceToHost, stream[i]);
    }

    for (int i = 0; i < 4; i++) {
        cudaFreeAsync(gpuArr1Pointers[i], stream[i]);
        cudaFreeAsync(gpuArr2Pointers[i], stream[i]);
        cudaStreamDestroy(stream[i]);
    }
    free(allignedArr);
    const auto endCPU = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();
}


float sequentialTest(int* arr, int n) {
    // we return the cpu time.
    float cpuTime = 0;
    float gpuTime = 0;
    const auto startCPU = std::chrono::steady_clock::now();
    int* clone = (int*) malloc(n * sizeof(int));
```

```
    for (int i = 0; i < 4; i++) {
        memcpy(clone, arr, n * sizeof(int));
        int val = gpuAdvancedMaxReductionSequental(arr, n, 64, i);
        //printf("%d\n", val);
    }
    const auto endCPU = std::chrono::steady_clock::now();

    return std::chrono::duration<double, std::milli>(endCPU -
    ↪  startCPU).count();;

}

// array shuffle function
void shuffle(int* arr, int n) {
    // own version of shuffle
    for (int i = 0; i < n; i++) {
        int j = rand() % n;
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}



int main() {
    int testArray[] = {
        1, 3, 10, 2, 7, 2 ,1, 21, 20 ,2, 5, 1
    };

    // generate array starting at 0 counting to size-1
    int testArray2[N];
    for (int i = 0; i < N; i++) {
        testArray2[i] = (i % 1 );
    }
    int dst[4];
    for (int i = 0; i < 10; i++) {
        sequentialTest(testArray2, N);
        asyncTest(testArray2, N, 64, dst);
    }

    // "prime the gpu"


    // shuffle the array
    shuffle(testArray2, N);


    /*float time1, time2;
```

```
    int value = gpuAdvancedMaxReduction(testArray, 12, 16, &time1,
    ↪  &time2);

    printf("Max value: %d\n", value);*/
    //float cpuTimeSequential = sequentialTest(testArray2, N);
    //printf("Cpu time sequential: %f\n", cpuTimeSequential);
    printf("Array size;Time sequential;Time async\n");
    for (int n = 0; n < N; n += 256) {

        float timeAsync = 0;
        float timeSeq = 0;

        for (int j = 0; j < 50; j++) {
            timeAsync += asyncTest(testArray2, n, 64, dst);
            timeSeq += sequentialTest(testArray2, n);
        }
        printf("%d;%f;%f\n", n, timeSeq / 50, timeAsync / 50);


    }


    /*float time = asyncTest(testArray2, smolN, 64, dst);
    for(int i = 0 ; i < 4; i++) {
        printf("%d\n", dst[i]);
    }
    printf("Time: %f\n", time);

    float timeSeq = sequentialTest(testArray2, smolN);
    printf("Time seq:%f\n", timeSeq);*/
    return 0;
}
```